An Introduction to E-Commerce

with a Focus on Machine Learning, Deep Learning, and Artificial Intelligence

Lecture 08: Recommenders II: Deep Learning, Embeddings, and Retrieval

Dr. Haitham A. El-Ghareeb

Faculty of Computers and Information Sciences
Mansoura University
Egypt

March 23, 2026

Session plan (90 minutes)

1. From classical recommenders to deep retrieval
2. Modern recommender architecture: retrieval and ranking
3. Embeddings as learned representations
4. Two-tower retrieval models
5. Approximate nearest-neighbor retrieval
6. Hands-on lab: retrieval plus re-ranking

Learning objectives

By the end of this lecture, you should be able to:

- Explain why deep learning methods are useful for large-scale recommendation and retrieval problems in e-commerce.
- Distinguish candidate generation from re-ranking and understand why modern recommenders are often multi-stage systems.
- Describe embeddings, two-tower retrieval, sequence models, and session-aware recommendation at a system level.
- Analyze production concerns such as cold start, diversity, latency, and bias in deep recommender architectures.

From classical recommenders to deep retrieval

- extremely large catalogs,
- rapidly changing inventories,
- sparse user histories,
- multi-modal item information such as text and images,
- session behavior that changes within minutes,
- and tight latency requirements at serving time.

Modern recommender architecture: retrieval and ranking

- **Candidate generation:** Candidate generation or retrieval narrows a huge catalog to a much smaller set of promising items.
- **Re-ranking:** After candidate generation, a second-stage model or rule system orders the retrieved candidates more precisely.
- **Why multi-stage design matters:** retrieval prioritizes speed and coverage,; re-ranking prioritizes precision and business control.

Embeddings as learned representations

- **Why embeddings are powerful:** capture similarity in a continuous space,; support nearest-neighbor retrieval,
- **What can be embedded:** users or customer accounts,; products,
- **Interpretation:** An embedding dimension does not usually have a simple human-readable meaning.

Two-tower retrieval models

- **Basic structure:** one tower that maps user or context features to an embedding,; one tower that maps item features to an embedding.
- **Why two towers are useful:** Two-tower models are attractive because item embeddings can often be precomputed offline and indexed for fast approximate nearest-neighbor retrieval.
- **Typical inputs:** recent interaction history,; device and channel context,

Approximate nearest-neighbor retrieval

- **Why exact search is often too expensive:** If the platform must compare each request embedding against millions of item embeddings, exact retrieval may be too slow for production latency budgets.
- **Architectural implication:** embedding refresh cadence,; index rebuild strategy,

Sequence models and session-aware recommendation

- **Session behavior:** last viewed products,; order of category transitions,
- **Sequence model families:** recurrent models,; convolutional sequence models,
- **Why sequence models help:** stable preference from temporary intent,; exploration from purchase preparation,

Multi-modal recommendation

- **Text and semantic understanding:** Text encoders can help represent product meaning beyond exact attribute matching.
- **Image understanding:** visually similar item recommendation,; cold-start support for new products,
- **Fusion:** Modern item representations often combine ID-based, attribute-based, text-based, and image-based features.

Deep re-ranking

- **Why re-ranking differs from retrieval:** user-item interaction history,; session context,
- **Business role of re-ranking:** The re-ranker is often where business policy becomes explicit.

Training data and objectives

- **Positive and negative examples:** positive examples from clicks, carts, or purchases,; sampled negatives from unseen or skipped items,
- **Objective choices:** next-item prediction,; pairwise ranking losses,
- **Label quality:** As in earlier chapters, the label design is a business interpretation problem.

Cold start in deep recommendation

- **User cold start:** session behavior,; page context,
- **Item cold start:** For new items, deep models can use metadata and content features more effectively than classical ID-only approaches.
- **Catalog churn:** Commerce catalogs change frequently.

Diversity, exploration, and bias

- **Diversity:** Diversity can improve user experience by preventing repetitive lists and broadening discovery.
- **Exploration:** Exploration is necessary to learn about new items, uncertain user interests, and shifting trends.
- **Bias:** popularity bias,; seller-exposure bias,

Serving and latency constraints

- **Key serving questions:** Which features are available in real time?; Which embeddings can be precomputed?
- **Caching and freshness:** Caching can reduce latency, but stale caches can recommend unavailable or irrelevant items.
- **Fallback behavior:** popularity-based lists,; category best-sellers,

Evaluation beyond offline metrics

- **Why deep models can mislead offline:** latency increases and user experience degrades,; popularity bias worsens,
- **Online considerations:** click-through and add-to-cart lift,; incremental conversion,

A practical deep recommender stack

- event collection and identity stitching,
- a feature store or equivalent feature pipelines,
- model training jobs,
- embedding generation,
- ANN index building,
- retrieval serving,

Hands-on lab: retrieval plus re-ranking

- **Lab scenario:** Students are given a dataset with user interaction histories, product metadata, and timestamps.
- **Lab tasks:** Train or simulate product embeddings from interaction and metadata signals.; Build a simple candidate retrieval mechanism using embedding similarity.
- **Expected learning outcome:** By the end of the lab, students should be able to explain why a high-quality recommender is a pipeline and not just a neural model.

Managerial implications

- richer representation learning,
- very large-scale retrieval,
- multi-modal item understanding,
- session-aware adaptation.